

Informe final - Proyecto de IA Conversacional

Asistente de soporte IT por voz con Dialogflow ES, FastAPI, RAG, Qdrant y Ollama

Proyecto MASTER - Practicas

Junio 2026

Índice

1	Resumen ejecutivo	2
2	Objetivos del proyecto	2
3	Arquitectura general	3
4	Componentes principales	4
4.1	Dialogflow ES	4
4.2	Entidad personalizada <code>codigo_error</code>	4
4.3	FastAPI	5
4.4	Cliente de Dialogflow desde FastAPI	5
4.5	Webhook de Dialogflow	5
5	Sistema RAG	6
5.1	Base vectorial Qdrant	6
5.2	Modelo de embeddings	6
5.3	Recuperacion documental	6
6	Modelo de lenguaje local	7
6.1	Pruebas con Llama	7
6.2	Cambio a Qwen	7
6.3	Parametros de generacion	7
7	Flujo de voz	7
7.1	STT externo	7
7.2	TTS externo	8
8	Problemas encontrados y soluciones	8
8.1	Dialogflow no llamaba al webhook	8
8.2	Respuesta <code>Not available</code> en Dialogflow	8
8.3	Descarga repetida del modelo de embeddings	8
8.4	Codigos sin guion procedentes de voz	8
8.5	Bloqueo circular en el flujo de voz	9
8.6	Diferencias entre <code>dialogflow_client.py</code> y <code>dialogflow_webhook.py</code>	9

9 Configuración final recomendada	9
9.1 Variables de entorno	9
9.2 Docker Compose	9
10 Comandos de ejecución	10
10.1 Arranque	10
10.2 Comprobación de contenedores	10
10.3 Descarga y calentamiento del modelo	10
10.4 Ingesta documental	10
10.5 Comprobación de Qdrant	10
11 Pruebas realizadas	10
11.1 Prueba local del webhook	10
11.2 Prueba local sin parametro <code>codigo_error</code>	11
11.3 Prueba desde Dialogflow Console	11
11.4 Prueba de voz en modo debug	11
11.5 Prueba de voz con salida de audio	12
12 Entregables	12
13 Guión recomendado para el video demo	12
13.1 1. Arranque del sistema	12
13.2 2. Dialogflow y webhook	13
13.3 3. Prueba local del webhook	13
13.4 4. Prueba por voz	13
13.5 5. Salida por voz	13
14 Conclusiones	13

1 Resumen ejecutivo

El proyecto desarrollado consiste en un asistente conversacional de soporte IT capaz de recibir consultas por voz, transcribirlas, interpretar la intención del usuario con Dialogflow ES, consultar documentación técnica mediante un sistema RAG y devolver una respuesta final también por voz.

El objetivo principal ha sido construir un flujo completo de IA conversacional que combine procesamiento de lenguaje natural, reconocimiento de voz, integración con un agente conversacional, búsqueda documental, base vectorial, modelo de lenguaje local y síntesis de voz. La solución final se ha orientado a un caso práctico de soporte técnico, con consultas sobre errores, procedimientos de backup, recuperación ante desastres y documentación IT.

Durante la implementación se realizaron varias optimizaciones para que el sistema funcionase de forma estable con Dialogflow ES. En particular, se sustituyó el modelo inicial `llama3.2:3b` por `qwen2.5:0.5b`, se redujo el contexto RAG a dos fragmentos de 700 caracteres, se configuró Uvicorn con dos workers y se añadió normalización de códigos técnicos procedentes de audio, por ejemplo BKP302 a BKP-302.

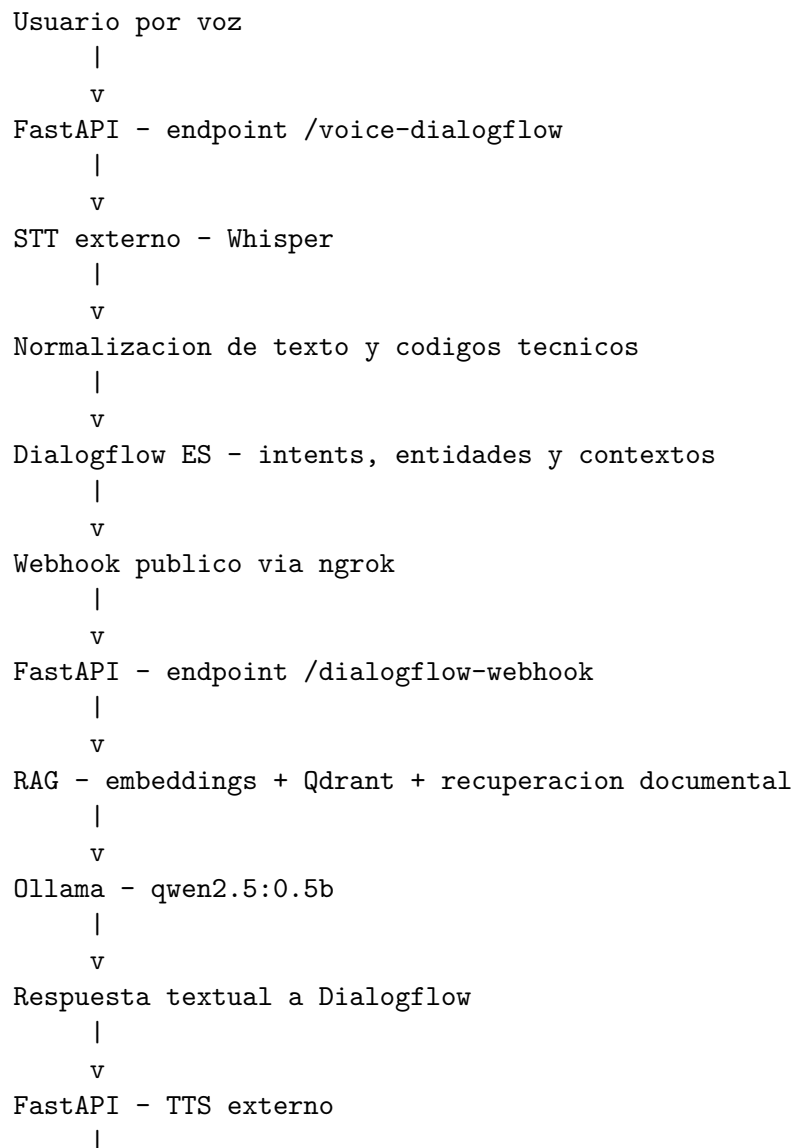
2 Objetivos del proyecto

Los objetivos funcionales y técnicos del proyecto son los siguientes:

- Crear un asistente conversacional orientado a soporte IT.
- Permitir consultas por voz y obtener respuestas tambien por voz.
- Usar Dialogflow ES para la deteccion de intenciones y extraccion de entidades.
- Usar STT externo a Dialogflow para convertir audio en texto.
- Usar TTS externo a Dialogflow para generar audio de respuesta.
- Integrar un webhook desarrollado con FastAPI.
- Implementar un sistema RAG con embeddings y base vectorial Qdrant.
- Usar un LLM local mediante Ollama para generar respuestas tecnicas.
- Incluir documentacion tecnica de prueba en la base de conocimiento.
- Entregar codigo, documentacion, exportacion del agente Dialogflow y video de demostracion.

3 Arquitectura general

La arquitectura final se organiza en varios componentes desacoplados, desplegados principalmente mediante Docker Compose.



Audio de respuesta

La solución usa Dialogflow ES como motor conversacional principal para clasificar la intención y extraer entidades. El conocimiento técnico no se codifica directamente en Dialogflow, sino que se consulta dinámicamente a través del webhook y del sistema RAG.

4 Componentes principales

4.1 Dialogflow ES

Dialogflow ES se usa para gestionar la capa conversacional. El agente contiene intents personalizados para saludar, pedir ayuda, consultar errores técnicos, consultar procedimientos, cancelar operaciones, corregir datos, volver a un tema anterior y finalizar la conversación.

Entre los intents definidos se encuentran:

- `saludo_usuario`
- `ayuda_general`
- `consultar_error_tecnico`
- `consultar_backup`
- `consultar_procedimiento_dr`
- `consultar_documento`
- `resumir_documentacion`
- `corregir_dato`
- `volver_tema_anterior`
- `cancelar_operacion`
- `despedida`

El intent más importante para la demo es `consultar_error_tecnico`, que permite preguntas como:

Que significa el error BKP-302?

Que significa el error BKP302?

Tengo el error SRV-1001

Busca información sobre DR-2024

4.2 Entidad personalizada `codigo_error`

Se utiliza una entidad personalizada para capturar códigos técnicos. Durante las pruebas se detectó que el reconocimiento de voz podía transcribir BKP-302 como BKP302, por lo que la entidad y la normalización del backend contemplan ambas variantes.

La expresión regular recomendada para reconocer códigos es:

`[A-Za-z]{2,5}-?[0-9]{3,4}`

Ejemplos reconocidos:

- BKP-302
- BKP302
- SRV-1001
- SRV1001
- DR-2024

- DR2024

4.3 FastAPI

FastAPI actua como backend principal de la solucion. Expone varios endpoints:

- `/health`: comprobacion simple del servicio.
- `/ask`: consulta directa al sistema RAG sin Dialogflow.
- `/dialogflow-webhook`: webhook llamado por Dialogflow.
- `/voice-dialogflow-debug`: prueba por voz con salida JSON para depuracion.
- `/voice-dialogflow`: flujo completo de voz con respuesta de audio.

El endpoint mas relevante para Dialogflow es `/dialogflow-webhook`. Recibe el payload de Dialogflow, extrae el intent, el texto y los parametros, y decide si debe responder con una respuesta estatica o llamar al RAG.

4.4 Cliente de Dialogflow desde FastAPI

El fichero `dialogflow_client.py` permite que FastAPI llame a Dialogflow mediante la API oficial. Esto se usa en el flujo de voz:

FastAPI recibe audio -> Whisper transcribe -> FastAPI llama a Dialogflow -> Dialogflow detecta

Este cliente se configuro para soportar agentes regionales, en este caso `europe-west2`. Por ello se construye la sesion con la ruta:

```
projects/<PROJECT_ID>/locations/europe-west2/agent/sessions/<SESSION_ID>
```

Tambien se ajusto la lectura de la respuesta para contemplar tanto `fulfillment_text` como `fulfillment_messages`, ya que en algunas respuestas de Dialogflow el texto puede llegar en una u otra estructura.

4.5 Webhook de Dialogflow

El fichero `dialogflow_webhook.py` implementa la logica que se ejecuta cuando Dialogflow llama al backend. Este flujo es el inverso al cliente anterior:

Dialogflow detecta intent -> Dialogflow llama a FastAPI -> FastAPI ejecuta RAG -> FastAPI devuelve

El webhook devuelve siempre un JSON compatible con Dialogflow:

```
{
  "fulfillmentText": "Respuesta generada por el sistema",
  "source": "fastapi-rag-webhook"
}
```

Para consultas RAG, el webhook usa preferentemente el parametro `codigo_error` extraido por Dialogflow. Si Dialogflow no lo envia, FastAPI intenta extraerlo directamente del `queryText`. Esto hace el sistema mas robusto para consultas procedentes de voz.

5 Sistema RAG

5.1 Base vectorial Qdrant

Qdrant se usa como base vectorial para almacenar fragmentos de documentos tecnicos. La coleccion utilizada es:

```
it_knowledge
```

Cada punto almacenado contiene:

- vector del fragmento documental;
- texto del fragmento;
- fuente del documento;
- pagina, si aplica;
- metadatos adicionales.

5.2 Modelo de embeddings

Para generar embeddings se usa:

```
sentence-transformers/all-MiniLM-L6-v2
```

Durante las pruebas se observo que este modelo podia descargarse de nuevo al reconstruir contenedores si la cache no estaba persistida. Por ello se recomienda montar una cache de Hugging Face persistente en Docker Compose:

```
volumes:  
  - hf_cache:/root/.cache/huggingface
```

5.3 Recuperacion documental

La recuperacion se hace codificando la pregunta del usuario con el mismo modelo de embeddings y consultando Qdrant por similitud coseno.

La configuracion final optimizada es:

```
RAG_TOP_K=2  
RAG_MIN_SCORE=0.40
```

Ademas, para reducir latencia y evitar respuestas inestables, el contexto enviado al LLM se limita a:

```
2 chunks maximo  
700 caracteres por chunk
```

La construccion del contexto final sigue esta idea:

```
Fuente: <documento> pagina: <pagina>  
<texto del fragmento limitado a 700 caracteres>
```

Esta decision mantiene el flujo RAG completo, pero evita enviar demasiado texto al modelo generativo.

6 Modelo de lenguaje local

6.1 Pruebas con Llama

Inicialmente se probaron modelos Llama mediante Ollama, en concreto:

- llama3.2:3b
- llama3.2:1b

Ambos modelos funcionaban correctamente en pruebas locales, pero en el flujo completo con Dialogflow ES podían tardar demasiado. El problema no era que FastAPI no pudiera esperar, sino que Dialogflow ES tiene un tiempo práctico limitado para recibir la respuesta del webhook. Si la respuesta del RAG llegaba tarde, Dialogflow mostraba respuestas como `Not available`.

6.2 Cambio a Qwen

Para mejorar la estabilidad del flujo se adoptó:

`OLLAMA_MODEL=qwen2.5:0.5b`

Este modelo permitió reducir la latencia manteniendo el flujo completo:

Dialogflow -> FastAPI -> embeddings -> Qdrant -> Ollama/Qwen -> Dialogflow

No se sustituyó el RAG por respuestas fijas. La optimización consistió en usar un LLM local más ligero y reducir el contexto de entrada.

6.3 Parametros de generacion

Para hacer la respuesta más estable y menos variable entre ejecuciones, se ajustaron los parámetros de generación:

```
{
  "temperature": 0,
  "top_p": 0.1,
  "top_k": 1,
  "seed": 42,
  "num_predict": 80,
  "num_ctx": 1024
}
```

Con estos parámetros, el modelo reduce la aleatoriedad y tiende a generar respuestas más repetibles.

7 Flujo de voz

7.1 STT externo

El reconocimiento de voz se realiza fuera de Dialogflow usando Whisper. El endpoint `/voice-dialogflow-debug` recibe un fichero de audio, lo guarda temporalmente y lo transcribe.

Durante las pruebas se detectó este comportamiento:

```
{
  "transcription_raw": "que significa el error BKP302."
}
```

El sistema normaliza despues el texto para convertir codigos sin guion en codigos normalizados:

```
{
  "transcription": "que significa el error BKP-302."
}
```

Esta normalizacion es importante porque los documentos tecnicos y el RAG pueden contener los codigos con guion.

7.2 TTS externo

La respuesta final se convierte en audio mediante un servicio TTS externo a Dialogflow. Para mejorar la calidad de la voz de salida se recomienda usar una voz neural, por ejemplo mediante **edge-tts** con una voz en espanol como **es-ES-AlvaroNeural** o **es-ES-ElviraNeural**.

El flujo de salida por voz es:

Respuesta textual -> TTS -> archivo de audio -> usuario

8 Problemas encontrados y soluciones

8.1 Dialogflow no llamaba al webhook

Inicialmente la consola de Dialogflow devolvia respuestas estaticas como:

Voy a buscar informacion sobre ese codigo tecnico.

La causa era que el webhook no estaba activado correctamente en el intent o existian respuestas estaticas en **Responses**. Se soluciono activando **Enable webhook call for this intent**, configurando la URL de ngrok en Fulfillment y eliminando respuestas estaticas que interferian.

8.2 Respuesta Not available en Dialogflow

Dialogflow devolvia **Not available** cuando el webhook tardaba demasiado o no devolvia un **fulfillmentText** valido. Se diagnostico mediante ngrok y logs de FastAPI.

Las medidas aplicadas fueron:

- reducir el modelo LLM a **qwen2.5:0.5b**;
- limitar el contexto RAG a dos chunks;
- limitar cada chunk a 700 caracteres;
- limitar **num_predict**;
- configurar parametros deterministas;
- calentar el modelo antes de la demo.

8.3 Descarga repetida del modelo de embeddings

Se observo que el modelo de embeddings podia descargarse repetidamente, aproximadamente 139 MB. La solucion fue persistir la cache de Hugging Face con un volumen Docker.

8.4 Codigos sin guion procedentes de voz

Whisper transcribia BKP-302 como BKP302. Aunque Dialogflow podia entrenarse para aceptar ambas variantes, se implemento normalizacion en FastAPI para convertir automaticamente:

BKP302 -> BKP-302
SRV1001 -> SRV-1001
DR2024 -> DR-2024

Ademas, el webhook extrae el codigo desde el texto si Dialogflow no envia el parametro.

8.5 Bloqueo circular en el flujo de voz

El flujo de voz produjo un problema curioso: FastAPI llamaba a Dialogflow y Dialogflow llamaba de vuelta al webhook del mismo FastAPI. Con un solo worker, el proceso podia quedar esperando la respuesta de Dialogflow mientras Dialogflow esperaba la respuesta del webhook.

La solucion fue ejecutar Uvicorn con dos workers:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2
```

Esto permite atender simultaneamente la peticion de voz y la llamada entrante del webhook.

8.6 Diferencias entre dialogflow_client.py y dialogflow_webhook.py

Se diferencian dos ficheros clave:

- `dialogflow_client.py`: permite que FastAPI llame a Dialogflow. Se usa en el flujo de voz despues de transcribir el audio.
- `dialogflow_webhook.py`: permite que Dialogflow llame a FastAPI. Se usa cuando Dialogflow necesita una respuesta dinamica mediante webhook.

En el flujo completo de voz intervienen ambos.

9 Configuracion final recomendada

9.1 Variables de entorno

```
QDRANT_URL=http://qdrant:6333
QDRANT_COLLECTION=it_knowledge
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
OLLAMA_URL=http://ollama:11434
OLLAMA_MODEL=qwen2.5:0.5b
RAG_TOP_K=2
RAG_MIN_SCORE=0.40
STT_MODEL=base
DIALOGFLOW_PROJECT_ID=asistente-soporte-it-pbdv
DIALOGFLOW_LANGUAGE_CODE=es
DIALOGFLOW_LOCATION=europe-west2
GOOGLE_APPLICATION_CREDENTIALS=/app/credentials/dialogflow-service-account.json
```

9.2 Docker Compose

Se recomienda que el servicio FastAPI se ejecute con dos workers y que se persista la cache de Hugging Face:

```
command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2
```

```
volumes:
  - ./data:/app/data
  - ./credentials:/app/credentials:ro
  - hf_cache:/root/.cache/huggingface
```

Y al final:

```
volumes:
  hf_cache:
```

10 Comandos de ejecucion

10.1 Arranque

```
docker compose up -d --build --force-recreate
```

10.2 Comprobacion de contenedores

```
docker compose ps
```

Deben estar activos:

```
api
qdrant
ollama
```

10.3 Descarga y calentamiento del modelo

```
docker compose exec ollama ollama pull qwen2.5:0.5b
docker compose exec ollama ollama run qwen2.5:0.5b "Responde hola"
```

10.4 Ingesta documental

```
docker compose exec api env PYTHONPATH=/app python ingestion/ingest_documents.py
```

No es necesario reingestar si solo se recrea el contenedor de API. Si se ejecuta `docker compose down -v`, se borran volúmenes y podría ser necesario reingestar.

10.5 Comprobacion de Qdrant

```
curl http://localhost:6333/collections
curl http://localhost:6333/collections/it_knowledge
```

11 Pruebas realizadas

11.1 Prueba local del webhook

```
curl -X POST http://localhost:8000/dialogflow-webhook \
  -H "Content-Type: application/json" \
  --data @- <<'JSON'
{
  "queryResult": {
```

```

    "queryText": "que significa el error BKP-302.",
    "intent": {"displayName": "consultar_error_tecnico"},
    "parameters": {"codigo_error": "BKP-302"}
  }
}
JSON

```

Resultado esperado:

```

{
  "fulfillmentText": "El codigo tecnico BKP-302 se refiere a...",
  "source": "fastapi-rag-webhook"
}

```

11.2 Prueba local sin parametro codigo_error

```

curl -X POST http://localhost:8000/dialogflow-webhook \
  -H "Content-Type: application/json" \
  --data @- <<'JSON'
{
  "queryResult": {
    "queryText": "que significa el error BKP-302.",
    "intent": {"displayName": "consultar_error_tecnico"},
    "parameters": {}
  }
}
JSON

```

Esta prueba valida que FastAPI puede extraer el codigo desde el texto si Dialogflow no lo envia como parametro.

11.3 Prueba desde Dialogflow Console

Pregunta:

que significa el error BKP302

Resultado esperado:

El error BKP-302 se refiere a un fallo en la conexion con el repositorio de backup...

En ngrok debe verse:

POST /dialogflow-webhook 200 OK

11.4 Prueba de voz en modo debug

```

curl -X POST http://localhost:8000/voice-dialogflow-debug \
  -F "file=@$HOME/master/PRACTICAS/SPEECH/pregunta1.m4a"

```

Resultado esperado:

```

{
  "transcription_raw": "que significa el error BKP302.",

```

```

    "transcription": "que significa el error BKP-302.",
    "intent": "consultar_error_tecnico",
    "answer": "El error BKP-302...",
    "session_id": "..."
}

```

11.5 Prueba de voz con salida de audio

```

curl -X POST http://localhost:8000/voice-dialogflow \
  -F "file=@$HOME/master/PRACTICAS/SPEECH/pregunta1.m4a" \
  --output respuesta_dialogflow.mp3

```

```
afplay respuesta_dialogflow.mp3
```

12 Entregables

La entrega recomendada se organiza así:

```

ENTREGA_PRACTICA_IA_CONVERSACIONAL/
|-- informe_practica_ia_conversacional.pdf
|-- video_demo.mp4
|-- proyecto_ia_conversacional.zip
|-- dialogflow_es_agente_soporte_it.zip
`-- documentacion_entrega.zip

```

El enunciado permite entregar el código como ZIP, siempre que incluya README con instrucciones claras. GitHub puede usarse como opción adicional, pero no es obligatorio si se entrega el ZIP completo.

No deben incluirse credenciales reales en la entrega:

```

.env
credentials/dialogflow-service-account.json
__pycache__/
venv/
.cache/
.DS_Store

```

13 Guion recomendado para el video demo

La demo debe durar menos de cinco minutos y mostrar el funcionamiento completo.

13.1 1. Arranque del sistema

Mostrar:

```

docker compose up -d --build --force-recreate
docker compose ps

```

Explicar que se levantan FastAPI, Qdrant y Ollama.

13.2 2. Dialogflow y webhook

Mostrar Dialogflow Console con la pregunta:

que significa el error BKP302

Mostrar que detecta el intent `consultar_error_tecnico` y que responde mediante webhook.

Mostrar ngrok con:

POST /dialogflow-webhook 200 OK

13.3 3. Prueba local del webhook

Ejecutar el curl contra /dialogflow-webhook y mostrar el `fulfillmentText`.

13.4 4. Prueba por voz

Ejecutar:

```
curl -X POST http://localhost:8000/voice-dialogflow-debug \
  -F "file=@$HOME/master/PRACTICAS/SPEECH/pregunta1.m4a"
```

Mostrar la transcripcion original, la transcripcion normalizada, el intent y la respuesta.

13.5 5. Salida por voz

Ejecutar:

```
curl -X POST http://localhost:8000/voice-dialogflow \
  -F "file=@$HOME/master/PRACTICAS/SPEECH/pregunta1.m4a" \
  --output respuesta_dialogflow.mp3
```

afplay respuesta_dialogflow.mp3

14 Conclusiones

El proyecto cumple el objetivo de implementar un asistente conversacional de soporte IT por voz con Dialogflow ES, webhook propio, RAG documental, base vectorial y LLM local.

La mayor dificultad tecnica no fue la construccion del flujo basico, sino lograr que el flujo completo fuese estable dentro de los tiempos practicos de Dialogflow ES. Para ello se realizaron optimizaciones de modelo, contexto, concurrencia y normalizacion de codigos tecnicos.

La version final mantiene el flujo completo real:

voz -> STT -> Dialogflow -> FastAPI -> RAG/Qdrant -> Ollama/Qwen -> Dialogflow -> TTS -> voz

El sistema queda preparado para ampliarse con nuevos documentos, nuevos codigos tecnicos, mejores voces TTS y despliegue en una infraestructura mas estable que ngrok para entornos productivos.